



POLITECHNIKA WROCŁAWSKA  
WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI

---

# **Algorytmy Grafowe**

Algorytmy i Złożoność Obliczeniowa – Projekt 2

---

Autor:

Michał Pazurek

Prowadzący zajęcia:

mgr inż. Damian Mroziński

Wrocław, 2026r.

# Spis Treści

|                                                    |    |
|----------------------------------------------------|----|
| 1. Cel projektu.....                               | 3  |
| 2. Wprowadzenie teoretyczne.....                   | 3  |
| 3. Reprezentacje grafów.....                       | 4  |
| 3.1. Macierz incydencji.....                       | 5  |
| 3.2. Lista sąsiedztwa (następników).....           | 6  |
| 4. Badane problemy.....                            | 6  |
| 4.1. Minimalne drzewo rozpinające (MST).....       | 6  |
| 4.2. Problem najkrótszej ścieżki (SP).....         | 7  |
| 4.3. Problem maksymalnego przepływu (MF).....      | 7  |
| 5. Badane algorytmy.....                           | 7  |
| 5.1. Algorytm Prima – Problem MST.....             | 7  |
| 5.2. Algorytm Kruskala – Problem MST.....          | 8  |
| 5.3. Algorytm Dijkstry – Problem SP.....           | 8  |
| 5.4. Algorytm Bellmana-Forda – Problem SP.....     | 9  |
| 5.5. Algorytm Forda-Fulkersona – Problem MF.....   | 10 |
| 6. Wyniki.....                                     | 10 |
| 6.1. Badanie A (wpływ rozmiaru grafu).....         | 10 |
| 6.2. Badanie B (wpływ gęstości grafu).....         | 13 |
| 6.3. Badanie działania na gotowych datasetach..... | 15 |
| 7. Analiza wyników.....                            | 16 |
| 7.1. Badanie A (wpływ rozmiaru grafu).....         | 16 |
| 7.2. Badanie B (wpływ gęstości grafu).....         | 18 |
| 7.3. Badanie działania na gotowych datasetach..... | 19 |
| 8. Przemyślenia i wnioski końcowe.....             | 20 |
| 9. Bibliografia.....                               | 21 |

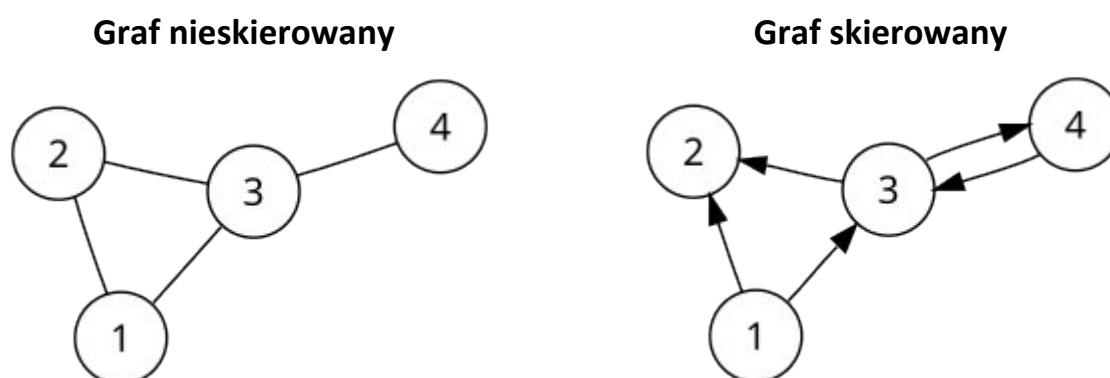
## 1. Cel projektu

Celem projektu było zapoznanie się z różnymi algorytmami grafowymi, ich implementacją oraz analizą efektywności w zależności od użytej reprezentacji grafu. Dodatkowo, projekt miał rozwinąć umiejętności skryptowania, mądrego planowania projektu oraz korzystania z gotowych datasetów.

## 2. Wprowadzenie teoretyczne

**Graf** to struktura matematyczna, która służy do przedstawiania i badania relacji między obiektami. Jest to zbiór wierzchołków połączonych krawędziami w taki sposób, że każda krawędź zaczyna się i kończy w którymś z wierzchołków [2]. Na potrzeby projektu rozważono dwa główne rodzaje grafów: grafy nieskierowane i skierowane.

Graf nieskierowany to taki graf, w którym krawędzie nie posiadają określonego kierunku - po jednej krawędzi  $E$  można dostać się z wierzchołka  $V_i$  do  $V_j$  oraz odwrotnie, z  $V_j$  do  $V_i$ . W grafie skierowanym natomiast, każda krawędź ma określony kierunek - po krawędzi  $E$  można przejść tylko z wierzchołka  $V_i$  do  $V_j$ . Różnice przedstawiono na Rysunek 1.



Rysunek 1: Różnica między grafem **skierowanym**, a **nieskierowanym**.

Źródła: [https://pl.wikipedia.org/wiki/Plik:Directed\\_graph\\_no\\_background.svg](https://pl.wikipedia.org/wiki/Plik:Directed_graph_no_background.svg)  
[https://pl.wikipedia.org/wiki/Plik:Undirected\\_graph\\_no\\_background.svg](https://pl.wikipedia.org/wiki/Plik:Undirected_graph_no_background.svg)

**Algorytmy grafowe** są jednymi z najważniejszych narzędzi w analizie i przetwarzaniu danych o złożonych relacjach. Wykorzystując struktury grafowe, takie jak węzły i krawędzie, algorytmy te pozwalają rozwiązać skomplikowane problemy związane z sieciami połączeń, takimi jak drogi, połączenia społeczne, systemy przesyłu energii, czy wiele innych, gdzie relacje między elementami można przedstawić za pomocą połączonych par [1].

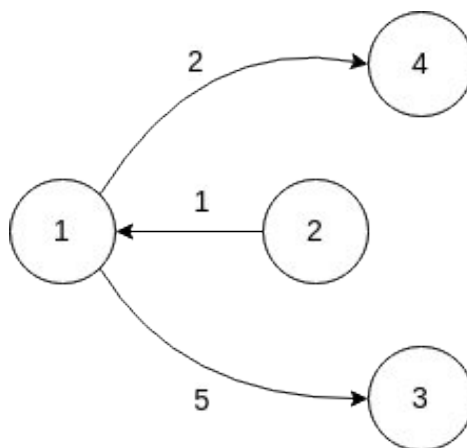
Jednymi z najpowszechniejszych przykładów użycia algorytmów grafowych są mapy z nawigacją GPS. Użycie grafów pozwala na wyznaczenie najkrótszej drogi z punktu A do punktu

B, z uwzględnieniem informacji takich jak stan i rodzaj drogi, zatory drogowe i wypadki, a w niektórych aplikacjach, także średnie zużycie paliwa na danej trasie. Oprócz tego, algorytmy grafowe stosowane są również w innych dziedzinach, takich jak:

- **Bioinformatyka** – analizowanie sieci metabolicznych i interakcji białek
- **Logistyka** – optymalizacja tras dostaw
- **Sztuczna inteligencja** – rozwiązywanie problemów w grach i robotyce
- **Cyberbezpieczeństwo** – identyfikacja nietypowych wzorców w ruchu sieciowym oraz analiza połączeń i interakcji między urządzeniami

### 3. Reprezentacje grafów

W celu przechowywania i przetwarzania grafów stosuje się odpowiednie struktury danych nazywane reprezentacjami grafowymi. W ramach projektu wykorzystano dwie reprezentacje: macierz incydencji oraz listę sąsiedztwa (następników). Obie te metody pozwalają na przechowywanie informacji o połączeniach między wierzchołkami, oraz wykonywanie na nich algorytmów, jednak różnią się sposobem organizacji danych. Na Rysunkach 3 i 4 przedstawiono przykład omawianych reprezentacji dla prostego grafu skierowanego przedstawionego na Rysunku 2.



Rysunek 2: Przykład grafu skierowanego

|                     |   | $E_k$ - krawędź |    |    |
|---------------------|---|-----------------|----|----|
|                     |   | 1               | 2  | 3  |
| $V_i$ - wierzchołek | 1 | 2               | 5  | -1 |
|                     | 2 | 0               | 0  | 1  |
|                     | 3 | 0               | -5 | 0  |
|                     | 4 | -2              | 0  | 0  |

Rysunek 3: **Macierz incydencji** dla grafu skierowanego z wagami

|                     |   | Lista sąsiadów dla $V_i$ |         |
|---------------------|---|--------------------------|---------|
| $V_i$ - wierzchołek | 1 | (4 : 2)                  | (3 : 5) |
|                     | 2 | (1 : 1)                  |         |
|                     | 3 | -                        |         |
|                     | 4 | -                        |         |

Rysunek 4: **Lista sąsiedztwa** dla grafu skierowanego z wagami

### 3.1. Macierz incydencji

**Macierz incydencji**, której przykład przedstawiono na Rysunku 3, to tablica dwuwymiarowa o wymiarach  $N \times K$ , gdzie  $N$  to liczba wierzchołków w grafie ( $N = |V|$ ), natomiast  $K$ , to liczba krawędzi ( $K = |E|$ ). Każdy wiersz odwzorowuje jeden wierzchołek grafu, a każda kolumna jedną krawędź [3]. Zawartość komórki tej macierzy zależy od przyjętej konwencji reprezentacji połączeń. Na potrzeby projektu, każda komórka może przybrać następujące wartości, określające relację wierzchołka  $V_i$  z krawędzią  $E_j$ :

- **0** – oznacza, że wierzchołek  $V_i$  nie jest częścią krawędzi  $E_j$
- **$W$**  – oznacza, że wierzchołek  $V_i$  jest początkiem krawędzi  $E_j$  o wadze  $W$
- **$-W$**  – oznacza, że wierzchołek  $V_i$  jest końcem krawędzi  $E_j$  o wadze  $W$

\* W przypadku grafu nieskierowanego zarówno wierzchołki początkowe, jak i końcowe oznaczone są przez  $W$ .

Wadami wybranej konwencji są: brak możliwości reprezentacji pętli – krawędzi zaczynających i kończących się w tym samym wierzchołku, a także reprezentacji krawędzi o ujemnych wagach. Dodatkowym problemem jest złożoność pamięciowa  $O(N \times K)$ , która powoduje szybki wzrost zapotrzebowania na pamięć RAM przy rosnącej liczbie wierzchołków i krawędzi, mimo że większość komórek przechowuje wartość 0.

### 3.2. Lista sąsiedztwa (następników)

**Lista sąsiedztwa** to tablica  $N$  elementów, gdzie  $N$  to liczba wierzchołków w grafie ( $N = |V|$ ). Każdy element tej tablicy reprezentuje jeden wierzchołek  $V_i$  i zawiera listę, w której znajdują się wierzchołki połączone z  $V_i$  (tzw. sąsiedzi) wraz z wagami tych połączeń (krawędzi). W celu graficznego przedstawienia elementów listy wykorzystujemy parę  $(V_j : W_{ij})$ , gdzie  $V_j$  to wierzchołek końcowy krawędzi, a  $W_{ij}$  to jej waga. Przykład listy sąsiedztwa dla grafu z Rysunku 2 przedstawia Rysunek 4.

Pod względem pamięciowym, lista sąsiedztwa jest jedną z najefektywniejszych reprezentacji grafowych, ponieważ przechowuje informacje tylko o istniejących połączeniach, bez konieczności rezerwacji miejsca dla nieistniejących połączeń. Dodatkowo, wyszukiwanie krawędzi jest również stosunkowo proste – mając wierzchołek startowy, wystarczy przejrzeć tylko jego następników aby stwierdzić, czy istnieje krawędź prowadząca z wierzchołka  $V_i$  do  $V_j$ . Niemniej jednak, w najgorszym przypadku przeszukanie całej listy sąsiadów może zająć sporo czasu gdyż jest to operacja liniowa. Mimo to, lista sąsiedztwa jest jedną z najczęściej stosowanych reprezentacji grafowych.

## 4. Badane problemy

W teorii grafów istnieje wiele problemów, których celem jest znalezienie najlepszego możliwego rozwiązania dla określonych warunków. W ramach tego projektu analizowane są trzy klasyczne problemy grafowe: minimalne drzewo rozpinające (MST), problem najkrótszej ścieżki (Shortest Path) oraz problem maksymalnego przepływu (Maximum Flow).

### 4.1. Minimalne drzewo rozpinające (MST)

Problem **minimalnego drzewa rozpinającego** polega na znalezieniu podzbioru krawędzi grafu nieskierowanego, który łączy wszystkie wierzchołki grafu bez tworzenia cykli i ma możliwie najmniejszą sumę wag krawędzi [4]. W prostszym języku: rozwiązaniem tego problemu jest połączenie wszystkich wierzchołków grafu w jak najtańszy sposób.

Problem ten jest często spotykany w sytuacjach, gdzie należy zaprojektować sieć połączeń w taki sposób, aby zminimalizować koszty. Przykładowo, jeśli firma telekomunikacyjna chce położyć okablowanie sieciowe, łączące wiele budynków w jedną sieć, będzie się starała zminimalizować koszty materiałów. Właśnie w tym przypadku problem MST pozwoli znaleźć takie połączenie wszystkich budynków, dla których koszty będą najniższe.

## 4.2. Problem najkrótszej ścieżki (SP)

Problem ten polega na znalezieniu ścieżki o minimalnym koszcie między dwoma wybranymi wierzchołkami w grafie. Koszt tej trasy może oznaczać odległość, czas przejazdu, opóźnienie lub inną miarę przypisaną do krawędzi jako jej waga.

Jest to jeden z najczęściej rozpatrywanych problemów grafowych. Możemy się z nim spotkać między innymi w nawigacji GPS, planowaniu tras transportowych, analizie sieci komputerowych oraz w grach komputerowych przy wyznaczaniu ścieżek poruszania się botów.

## 4.3. Problem maksymalnego przepływu (MF)

W problemie **maksymalnego przepływu** celem jest określenie najwyższej możliwej wartości „przepływu” jaki można przesłać przez sieć z wierzchołka źródłowego do docelowego, zwanego ujściem. Każda krawędź w grafie posiada określoną przepustowość, która ogranicza ilość przepływu mogącego przez nią przejść. Przepustowość pojedynczej ścieżki jest ograniczona do krawędzi o najmniejszej przepustowości znajdującą się na tej ścieżce. Z tego powodu znalezienie maksymalnego przepływu dla całego grafu nie jest takie proste – należy sprawdzić wiele możliwych dróg prowadzących od źródła do ujścia i odpowiednio rozdzielić pomiędzy nie przepływ tak, aby był największy.

Problem ten można wyobrazić sobie jako sieć rurociągową, gdzie każda rura, ze względu na swoją średnicę, może mieć różne maksymalne wartości przepływu wody. Oczywiście w takiej sieci woda może wybierać różne drogi w zależności od „zapełnienia” innych. Rozwiązaniem jest określenie maksymalnej wartości wody, którą można przesłać wszystkimi drogami bez przeciążania sieci. Właśnie w podobnych, rzeczywistych sytuacjach problem ten jest najbardziej zauważalny.

## 5. Badane algorytmy

### 5.1. Algorytm Prima – Problem MST

Pierwszym z badanych algorytmów jest algorytm Prima, rozwiązujący problem minimalnego drzewa rozpinającego dla grafu spójnego, nieskierowanego. Jest to algorytm zachłanny polegający na kolejnym dodawaniu krawędzi łączących sąsiadów wierzchołków tworzonego drzewa MST, wybierając przy tym te o najmniejszej wadze.

Schemat działania algorytmu wygląda następująco:

- 1) Tworzymy drzewo, do którego dodajemy dowolny wierzchołek grafu.

- 2) Tworzymy kolejkę priorytetową i dodajemy do niej wszystkich sąsiadów wierzchołków znajdujących się w drzewie MST – kluczem kolejki są wagi krawędzi łączących sąsiadów.
- 3) Do momentu dodania wszystkich wierzchołków do minimalnego drzewa rozpinającego powtarzamy następujące kroki:
  - pobieramy z kolejki wierzchołek, do którego koszt dojścia jest najniższy, i który nie znajduje się jeszcze w drzewie MST
  - dodajemy wierzchołek wraz z wagą do drzewa rozpinającego
  - dodajemy sąsiadów tego wierzchołka do kolejki priorytetowej

## 5.2. Algorytm Kruskala – Problem MST

Kolejnym algorytmem, rozwiązującym również problem minimalnego drzewa rozpinającego jest algorytm Kruskala. Podobnie jak w przypadku algorytmu Prima, działa on dla grafów spójnych i nieskierowanych i wyznacza drzewo rozpinające grafu o najniższej wadze. Reguła działania jest nieco inna – rozpoczynamy mając zbiór wszystkich krawędzi i po kolei dodajemy te, które mają najniższą wagę. Cały algorytm sprowadza się więc do następujących kroków:

- 1) z wierzchołków grafu tworzymy las, w którym każdy wierzchołek jest osobnym drzewem
- 2) tworzymy zbiór wszystkich krawędzi grafu i sortujemy je niemalejąco według wag
- 3) dopóki zbiór krawędzi nie jest pusty i nie otrzymaliśmy drzewa rozpinającego powtarzamy:
  - wybieramy i usuwamy ze zbioru krawędzi krawędź o najniższej wadze
  - jeśli krawędź ta połączy dwa **różne** drzewa z lasu wierzchołków, dodajemy ją do drzewa MST, jeśli nie – odrzucamy ją

Po zakończeniu tego algorytmu otrzymujemy drzewo rozpinające grafu o najniższej wadze.

## 5.3. Algorytm Dijkstry – Problem SP

Następnym algorytmem badanym w tym projekcie jest algorytm Dijkstry. Jest to pierwszy z algorytmów rozwiązujących problem najkrótszej ścieżki dla grafów o nieujemnych wagach. Sam sposób działania jest bardzo podobny do algorytmu MST Prima, z tą różnicą, że krawędzie są skierowane, zamiast drzewa używamy tablicy odległości, oraz dodatkowo przechowujemy tablicę poprzedników, w celu rekonstrukcji ścieżki. Teoretyczny opis działania wygląda następująco:

- 1) Tworzymy tablicę  $d$ , przechowującą odległości od źródła dla wszystkich wierzchołków grafu i ustawiamy  $d[start] = 0$ , oraz  $d[i] = \infty$
- 2) Tworzymy tablicę  $p$ , przechowującą poprzedników wierzchołków i ustawiamy  $p[i] = -1$
- 3) Tworzymy kolejkę priorytetową i dodajemy do niej wierzchołek startowy, wraz z wagą 0



4) Dopóki kolejka nie jest pusta powtarzamy następujące kroki:

- Pobieramy i usuwamy z kolejki wierzchołek o najniższej wadze
- Dla każdego sąsiada tego wierzchołka relaksujemy krawędź – jeśli odległość do sąsiada przez wierzchołek jest krótsza niż dotychczasowa, zamieniamy ją ustawiając  $d[v] = d[u] + w(u, v)$ , gdzie  $u$  to pobrany wierzchołek,  $v$  to sąsiad, a funkcja  $w(u, v)$  zwraca wagę krawędzi łączącej  $u$  z  $v$ . Dodatkowo, jeśli dokonaliśmy relaksacji, zmieniamy poprzednika ustawiając  $p[v] = u$
- Po udanej relaksacji dodajemy wszystkich sąsiadów wierzchołka  $u$  do kolejki priorytetowej

Po zakończeniu działania algorytmu otrzymujemy tablicę  $d$  z odległościami wszystkich wierzchołków od wierzchołka startowego oraz tablicę  $p$  z poprzednikami wszystkich wierzchołków.

## 5.4. Algorytm Bellmana-Forda – Problem SP

Drugim algorytmem rozwiązującym problem najkrótszej ścieżki jest algorytm Bellmana-Forda. W przeciwieństwie do algorytmu Dijkstry pozwala on na pracę z grafami zawierającymi krawędzie z ujemnymi wagami. Dodatkowo umożliwia on wykrywanie cykli o ujemnej wadze, dla których problem najkrótszej ścieżki nie posiada rozwiązania.

Algorytm opiera się na przeglądaniu wszystkich krawędzi grafu kilka razy i wykonywaniu relaksacji. Schemat działania algorytmu wygląda następująco:

- 1) Tworzymy tablicę  $d$  przechowującą odległości od źródła dla wszystkich wierzchołków grafu i ustawiamy  $d[start] = 0$  oraz  $d[i] = \infty$  dla pozostałych wierzchołków
- 2) Tworzymy tablicę  $p$  przechowującą poprzedników wierzchołków i ustawiamy  $p[i] = -1$
- 3) Powtarzamy następujące kroki  $|V| - 1$  razy, gdzie  $|V|$  to liczba wierzchołków grafu:
  - przeglądamy wszystkie krawędzie
  - dla każdej krawędzi wykonujemy relaksację – jeśli przejście przez analizowaną krawędź pozwala uzyskać krótszą ścieżkę, aktualizujemy odległość w tablicy  $d$  oraz poprzednika w tablicy  $p$
- 4) Po zakończeniu relaksacji wszystkich krawędzi wykonujemy dodatkowe przejście po wszystkich krawędziach grafu:
  - jeśli możliwe jest dalsze skrócenie którejś ze ścieżek, oznacza to, że wystąpił cykl o ujemnej wadze, więc informujemy o tym i kończymy algorytm

Po zakończeniu działania algorytmu otrzymujemy tablicę  $d$ , zawierającą odległości od wierzchołka startowego do wszystkich pozostałych wierzchołków oraz tablicę  $p$  umożliwiającą rekonstrukcję ścieżki.

## 5.5. Algorytm Forda-Fulkersona – Problem MF

Ostatnim badanym algorytmem jest algorytm Forda-Fulkersona, służący do rozwiązywania problemu maksymalnego przepływu. Celem algorytmu jest wyznaczenie największego możliwego przepływu pomiędzy wierzchołkiem źródłowym a tzw. ujściem, uwzględniając maksymalną przepustowość wszystkich krawędzi.

Oryginalny algorytm Forda-Fulkersona używa przeszukiwania w głąb (DFS, Depth First Search) do wyszukiwania ścieżek, natomiast w ramach projektu, w celu przyspieszenia działania zastosowano wariant wykorzystujący przeszukiwanie wszerz (BFS, Breadth First Search). Wersja ta jest znana też jako algorytm Edmondsa-Karpa.

Schemat działania algorytmu wygląda następująco:

- 1) Ustawiamy początkowy przepływ na wartość 0
- 2) Tworzymy graf residualny, którego przepustowości początkowo są równe przepustowościom krawędzi grafu wejściowego.
- 3) Dopóki istnieje ścieżka ze źródła do ujścia w grafie residualnym, powtarzamy następujące kroki:
  - za pomocą algorytmu BFS wyszukujemy ścieżkę powiększającą
  - wyznaczamy przepływ możliwy do przesłania tą ścieżką jako minimalną przepustowość spośród wszystkich krawędzi należących do ścieżki
  - zwiększamy wartość całkowitego przepływu o wyznaczoną wartość
  - aktualizujemy przepustowości w grafie residualnym, zmniejszając przepustowości wykorzystanych krawędzi oraz tworząc lub zwiększając przepustowości krawędzi przeciwnych
- 4) Algorytm kończy działanie w momencie, gdy nie istnieje już żadna ścieżka powiększająca prowadząca od źródła do ujścia.

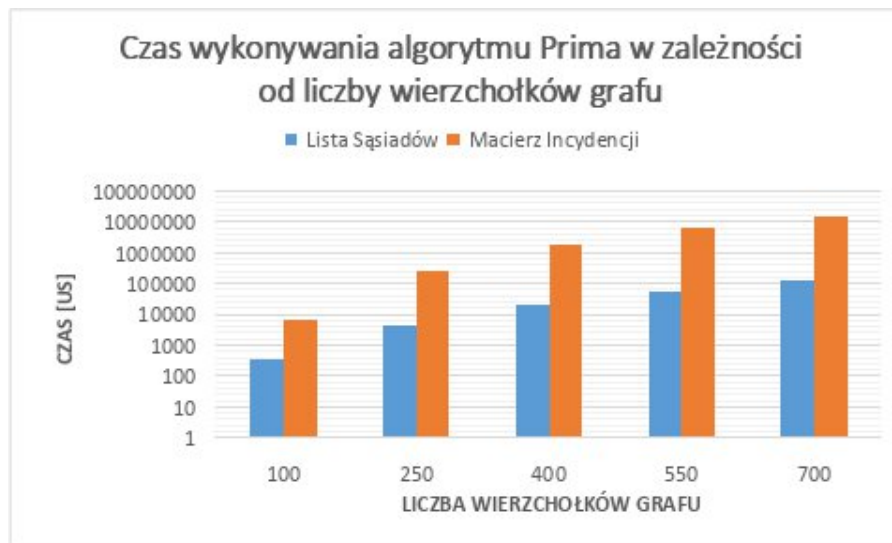
Po zakończeniu działania algorytmu otrzymujemy maksymalną wartość przepływu możliwą do przesłania pomiędzy źródłem a ujściem w grafie.

## 6. Wyniki

### 6.1. Badanie A (wpływ rozmiaru grafu)

Celem badania A było ustalenie wpływu rozmiaru grafu (liczby wierzchołków) o gęstości 50% na czas wykonywania poszczególnych algorytmów dla zadanych reprezentacji. Jako rozmiary grafów wybrano wartości 100, 250, 400, 550 oraz 700 wierzchołków. W przypadku problemów najkrótszej ścieżki oraz maksymalnego przepływu program przyjmuje za wierzchołek początkowy wierzchołek o indeksie 0, natomiast końcowy – wierzchołek o

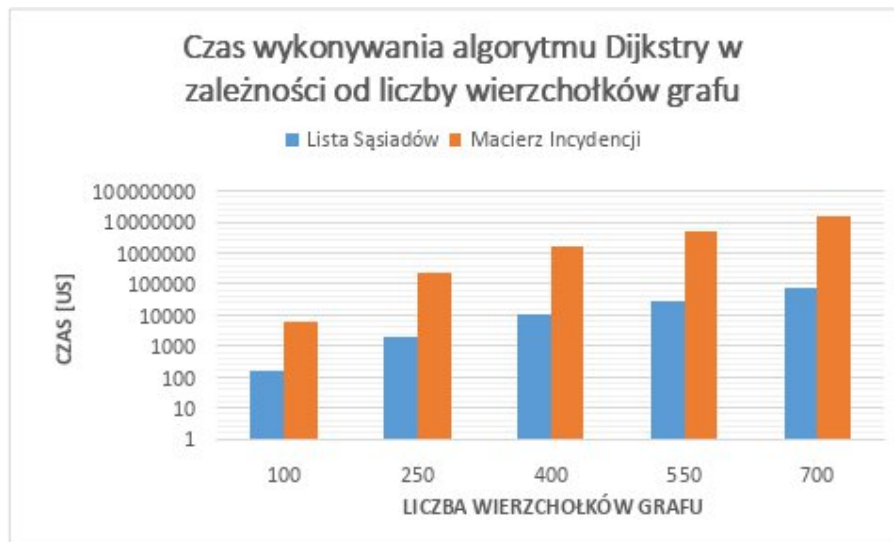
indeksie  $N - 1$ , gdzie  $N$  to liczba wszystkich wierzchołków grafu. Otrzymane wyniki przedstawiono na poniższych wykresach:



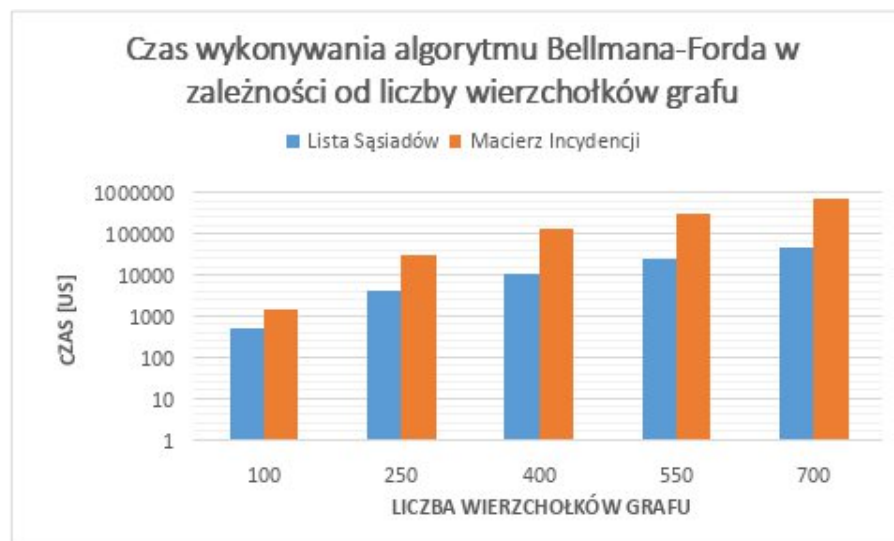
Rysunek 5: Czas wykonywania algorytmu **Prima** w zależności od liczby wierzchołków grafu



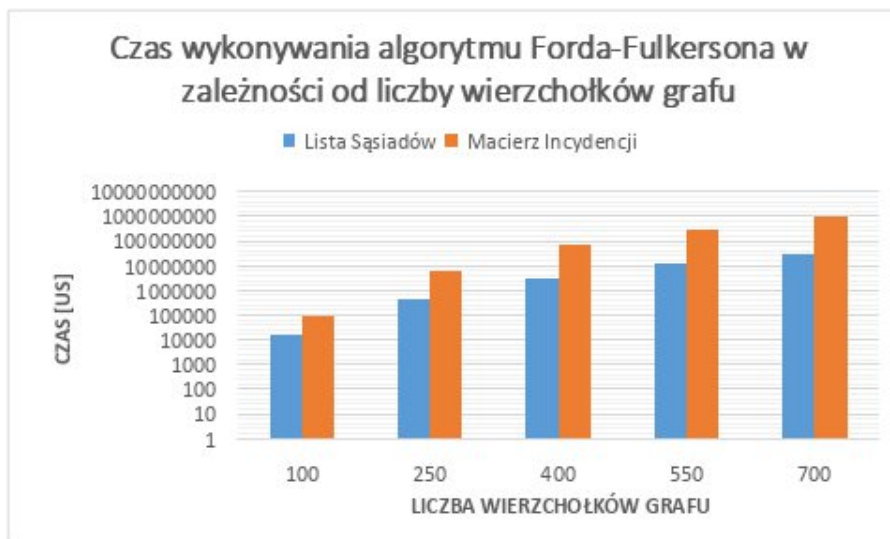
Rysunek 6: Czas wykonywania algorytmu **Kruskala** w zależności od liczby wierzchołków grafu



Rysunek 7: Czas wykonywania algorytmu **Dijkstry** w zależności od liczby wierzchołków grafu



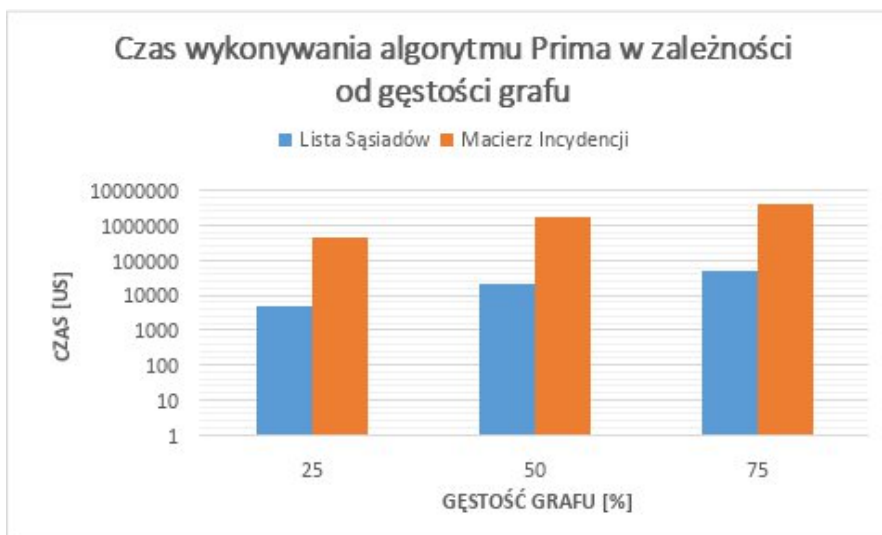
Rysunek 8: Czas wykonywania algorytmu **Bellmana-Forda** w zależności od liczby wierzchołków grafu



Rysunek 9: Czas wykonywania algorytmu **Forda-Fulkersona** w zależności od liczby wierzchołków grafu

## 6.2. Badanie B (wpływ gęstości grafu)

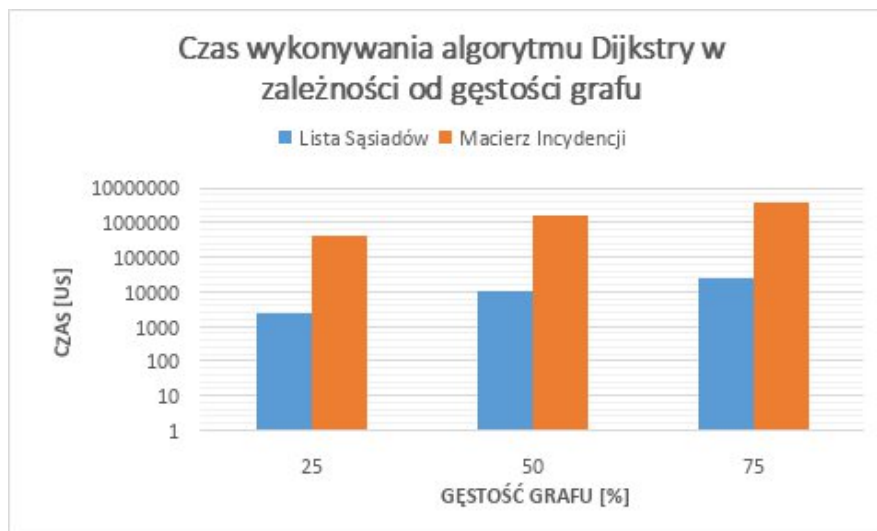
W ramach badania B należało sprawdzić zależność między gęstością grafu, a czasem wykonywania algorytmów na użytych strukturach. W tym celu wybrano środkowy rozmiar z badania A (400 wierzchołków) i przeanalizowano gęstości 25, 50 oraz 75%. Wyniki dla poszczególnych algorytmów przedstawiono na poniższych wykresach:



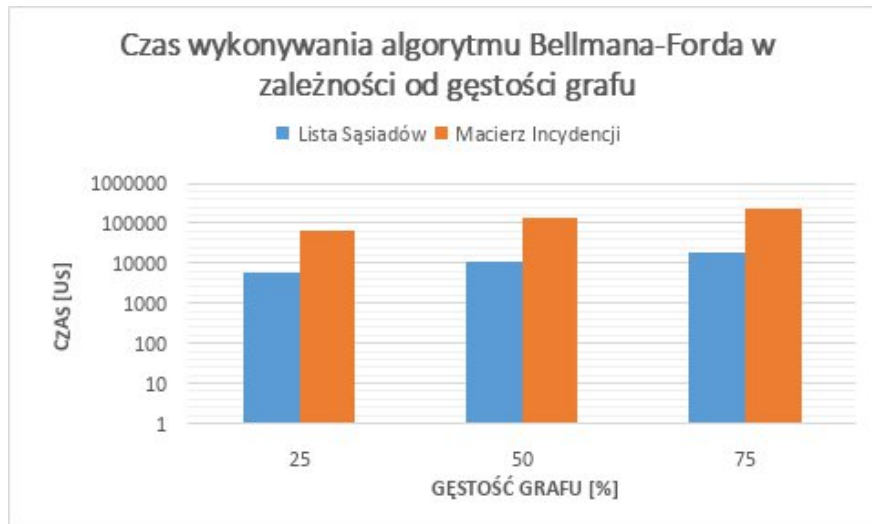
Rysunek 10: Czas wykonywania algorytmu **Prima** w zależności od gęstości grafu



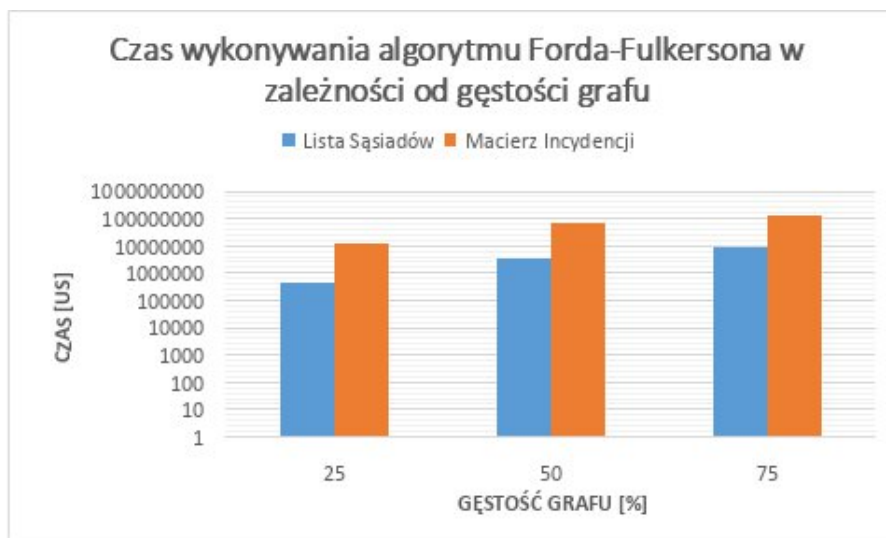
Rysunek 11: Czas wykonywania algorytmu **Kruskala** w zależności od gęstości grafu



Rysunek 12: Czas wykonywania algorytmu **Dijkstry** w zależności od gęstości grafu



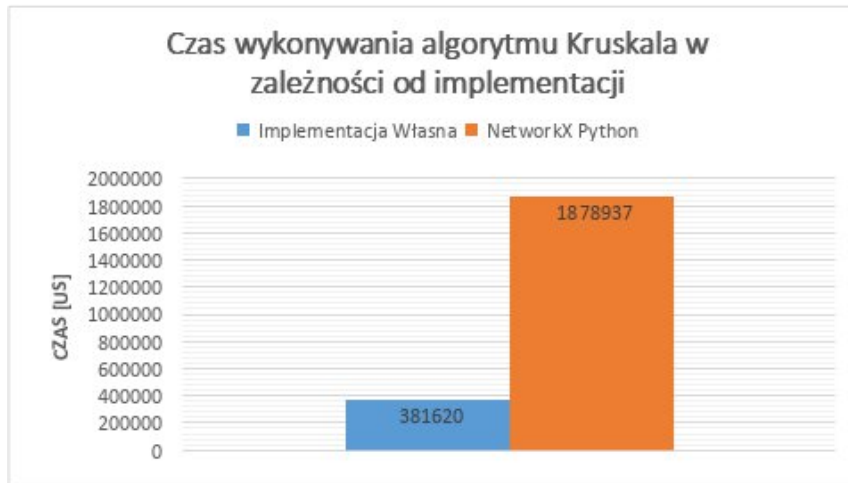
Rysunek 13: Czas wykonywania algorytmu **Bellmana-Forda** w zależności od gęstości grafu



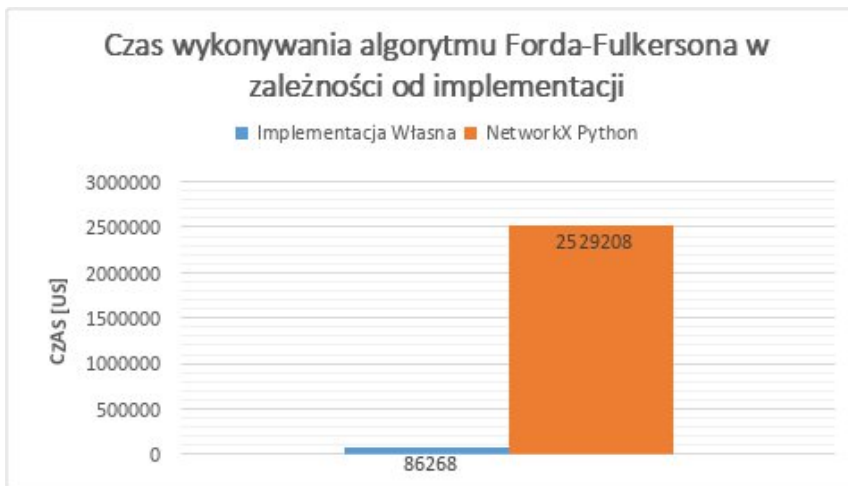
Rysunek 14: Czas wykonywania algorytmu **Forda-Fulkersona** w zależności od gęstości grafu

### 6.3. Badanie działania na gotowych datasetach

W ostatnim badaniu sprawdzono różnicę w czasie wykonywania algorytmów między własną implementacją w C++, a gotowym narzędziem jakim jest biblioteka NetworkX dla Pythona. W tym celu wybrano algorytm Kruskala, który we wcześniejszych badaniach poradził sobie najlepiej, oraz algorytm Forda-Fulkersona, który poradził sobie najgorzej. Jako dataset wybrano graf przedstawiający połączenia dróg w Nowym Jorku [11]. Wyniki przedstawiono poniżej:



Rysunek 15: Czas wykonywania algorytmu **Kruskala** w zależności od implementacji



Rysunek 16: Czas wykonywania algorytmu **Forda-Fulkersona** w zależności od implementacji

## 7. Analiza wyników

### 7.1. Badanie A (wpływ rozmiaru grafu)

Badanie A polegało na sprawdzeniu zależności między rozmiarem grafu (liczbą wierzchołków), a czasem wykonywania algorytmów grafowych. We wszystkich przeprowadzonych testach, niezależnie od analizowanego algorytmu oraz użytej struktury, wraz ze wzrostem liczby wierzchołków (a także krawędzi przez stałą gęstość 50%) rosta również liczba operacji, które musiały zostać wykonane przez algorytmy, a razem z tym, czas ich wykonywania.



Największy czas wykonania, oraz największe różnice między badanymi rozmiarami grafów osiągał algorytm Forda-Fulkersona. Już dla grafów o 100 wierzchołkach jego czas działania był zauważalnie większy niż w przypadku pozostałych badanych algorytmów, a wraz ze wzrostem liczby wierzchołków różnice te stawały się jeszcze bardziej widoczne. Wynika to z faktu, że algorytm ten wielokrotnie przeszukuje graf w poszukiwaniu ścieżki zwiększającej przepływ, co sprawia, że nawet dla stosunkowo małych zmian czas wykonywania potrafi znacznie urosnąć. Dodatkowo, można zauważyć, że jak w przypadku pozostałych algorytmów lista sąsiedztwa była znacznie szybsza niż macierz incydencji – przykładowo dla grafu o 700 wierzchołkach algorytm wykonywał się średnio 50s dla listy sąsiedztwa i aż około 17 minut dla macierzy incydencji. Fakt ten wynika ze sposobu znajdowania wszystkich sąsiadów wierzchołków. Dla macierzy incydencji należy przejrzeć wszystkie krawędzie, co ma złożoność  $O(|E|)$ , natomiast dla listy sąsiedztwa wystarczy przejść po sąsiadach wierzchołka, co określić można złożonością  $O(deg(v))$ , gdzie  $deg(v)$  to stopień (liczba sąsiadów) wierzchołka  $v$ .

Podobne zachowanie, tyle że w mniejszej skali, można było zauważyć również dla pozostałych algorytmów. Dodatkowo, największe różnice między reprezentacjami można było zauważyć w przypadku algorytmów Prima i Dijkstry. Już dla małych grafów były one zauważalne, a wraz ze wzrostem liczby wierzchołków stawały się ogromne. W przypadku badania dla 700 wierzchołków, czas działania dla macierzy incydencji był nawet o kilka rzędów wielkości większy niż dla listy sąsiedztwa, co można zauważyć na Rysunkach 5 i 7. Wynika to z faktu, że zarówno algorytm Prima, jak i algorytm Dijkstry bardzo często operują na sąsiadach wierzchołków, a jak opisano wcześniej, jest to dużo prostsze w przypadku listy sąsiedztwa niż w przypadku macierzy incydencji.

Nieco inaczej wyglądała sytuacja dla algorytmu Kruskala, którego wyniki przedstawione zostały na Rysunku 6. W tym przypadku wpływ rozmiaru grafu był również widoczny, jednak różnice pomiędzy reprezentacjami były znacznie mniejsze. Dla niewielkich grafów wyniki były do siebie bardzo podobne i dopiero przy większych grafach przewaga listy sąsiedztwa stawała się bardziej zauważalna. Działo się tak, ponieważ w tym algorytmie pobieramy krawędzie, a następnie używamy zbiorów rozłącznych (ang. disjoint set) do wyznaczenia drzewa MST, co znacznie przyspiesza działanie i ogranicza wpływ reprezentacji. Z tego też względu, algorytm Kruskala wykonywał się średnio szybciej niż drugi algorytm dla problemu MST – algorytm Prima. Różnice te można zauważyć porównując Rysunek 5 z Rysunkiem 6.

W przypadku algorytmu Bellmana-Forda sytuacja znów przedstawiała się podobnie – wraz ze wzrostem liczby wierzchołków oraz krawędzi czas wykonania stopniowo wzrastał dla obu reprezentacji, gdzie ponownie szybsza była lista sąsiedztwa, co przedstawione zostało na Rysunku 8.

Patrząc na wszystkie wyniki można więc zauważyć, że jak można by się spodziewać, wpływ rozmiaru grafu na czas działania algorytmów grafowych był znaczny i zgodny z przewidywaniami oraz teorią.

## 7.2. Badanie B (wpływ gęstości grafu)

W tym badaniu należało sprawdzić wpływ gęstości grafu na czas wykonywania badanych algorytmów. We wszystkich przeprowadzonych testach liczba wierzchołków została ustawiona na 400, a liczba krawędzi rostała wraz ze zmianą gęstości grafu z 25% na 50 oraz 75%. Jak można było się spodziewać, większe gęstości, a więc większe liczby krawędzi, powodowały wydłużenie czasu działania wszystkich algorytmów, niezależnie od użytej struktury.

Pierwszym przykładem jest przedstawiony na Rysunku 10 algorytm Prima. Podobnie jak w przypadku badania A, wraz ze wzrostem gęstości grafu rosła liczba sąsiadów, których należało sprawdzić podczas budowania drzewa MST. Z tego też względu, różnice między reprezentacjami były również zauważalne, ponieważ, jak wytłumaczono wcześniej, lista sąsiedztwa pozwala na szybsze przeglądanie sąsiadów danego wierzchołka niż macierz incydencji.

Bardzo zbliżone wyniki można było zaobserwować dla algorytmu Dijkstry przedstawionego na Rysunku 12. W tym przypadku średnie czasy wykonywania algorytmu były prawie identyczne z czasami wykonywania algorytmu Prima, gdyż sposób implementacji tych algorytmów jest bardzo podobny – oba używają kolejek priorytetowych wraz z sąsiadami wierzchołków i wagami łączących je krawędzi, co sprawiło, że ich wyniki są niemal identyczne.

Znów nieco inaczej wyglądała sytuacja dla algorytmu Kruskala, czego w sumie można się już było spodziewać. Jak można zauważyć na Rysunku 11, tutaj także wzrost czasu działania wraz ze wzrostem gęstości grafu był widoczny, jednak różnice między reprezentacjami były dużo mniejsze niż dla innych algorytmów. Jak wytłumaczono wcześniej, działa się tak dlatego, że głównym etapem algorytmu jest sortowanie krawędzi, które później przechowywane są już w tablicy dynamicznej. Dodatkowo, wierzchołki pobierane są z już pobranych krawędzi i dodawane do zbioru rozłącznego, co redukuje wpływ struktury tylko do jednorazowego pobrania wszystkich krawędzi.

Dużym zaskoczeniem był algorytm Bellmana-Forda. Oczywiście znów wraz ze wzrostem gęstości rósł czas wykonywania jednak był on średnio niższy niż dla algorytmu Dijkstry, który rozwiązuje ten sam problem, co można zauważyć porównując Rysunki 12 i 13. Zgodnie z teorią algorytm Bellmana-Forda powinien być wolniejszy niż Dijkstry jednak w tym przypadku było na odwrót. Stało się tak, ponieważ trzymano się wersji algorytmu Dijkstry przedstawionej na ćwiczeniach, która jednak okazała się nie być najoptymalniejszą wersją w realnym zastosowaniu – ciągłe pobieranie sąsiadów było znacznie wolniejsze niż jednorazowe pobranie krawędzi w przypadku algorytmu Bellmana-Forda, a według źródeł internetowych można było jednorazowo wstawić wszystkie wierzchołki do kolejki, co teoretycznie powinno przyspieszyć wykonywanie algorytmu Dijkstry. Niestety zabrakło już czasu żeby to zaimplementować i przetestować.

Wracając do analizy wyników, największe czasy wykonania osiągał znów algorytm Forda-Fulkersona, co w sumie nie było dużym zaskoczeniem. Znów, większa gęstość przekłada się na większą liczbę krawędzi, a to znów na większą liczbę ścieżek zwiększających przepływ, które należy sprawdzić. Wszystko to sprawia, że różnice między kolejnymi gęstościami znacznie pogarszają czas wykonywania tego algorytmu.

Podsumowując, gęstość grafu ma również znaczny wpływ na czas wykonywania na nim algorytmów. Wiąże się to z tym, że graf o większej gęstości ma więcej krawędzi, a więc każdy wierzchołek może mieć większą liczbę sąsiadów, co ważne jest w przypadku algorytmów Prima oraz Dijkstry, a także sama liczba krawędzi pogarsza czas wykonywania algorytmów takich jak algorytm Kruskala, Bellmana-Forda czy przede wszystkim Forda-Fulkersona.

### **7.3. Badanie działania na gotowych datasetach**

Było to ostatnie do przeprowadzenia badanie, które polegało na porównaniu własnej implementacji algorytmów grafowych z gotowym narzędziem. W tym celu napisano skrypt w Pythonie z użyciem biblioteki NetworkX do przetwarzania grafów. Testy przeprowadzono na rzeczywistym datasetcie przedstawiającym połączenia dróg w mieście Nowy Jork w USA. Źródłem grafu był dataset pobrany ze strony projektu DIMACS Challenge [11]. Jako że dane nie odpowiadały wymaganemu formatowi, użyto skryptu napisanego w Pythonie do najpierw posortowania wierzchołków, a następnie zmniejszenia każdego wierzchołka o 1 aby wymusić indeksowanie od 0. Po tych zmianach przystąpiono do badania, które pozwoliło sprawdzić nie tylko poprawność algorytmów, ale też ich wydajność w rzeczywistych sytuacjach.

Jak można zauważyć na Rysunku 15, w przypadku algorytmu Kruskala, własna implementacja osiągnęła czas około 382 tys.  $\mu$ s, natomiast NetworkX potrzebował około 1,88 mln  $\mu$ s. Oznacza to, że własna, najpewniej nieoptymalna implementacja w C++ była około 5 razy szybsza od dostępnej publicznie biblioteki Pythona. Różnica ta jest dosyć duża, jednak nie aż tak jak w przypadku algorytmu Forda-Fulkersona przedstawionego na Rysunku 16, gdzie własna implementacja osiągnęła czas około 86 tys.  $\mu$ s, natomiast biblioteka NetworkX potrzebowała ponad 2,5 mln  $\mu$ s, co przekłada się na około 30 razy lepszy czas dla C++. Wynika to najprawdopodobniej z tego, że w przypadku algorytmu Kruskala sortujemy krawędzie, gdzie gotowe rozwiązanie może w teorii używać szybszych algorytmów niż użyty w projekcie Mergesort. Dodatkowym aspektem wpływającym na spowolnienie w obu przypadkach jest też sam fakt użycia Pythona, który mimo wszystko sam z siebie jest dużo wolniejszy niż C++, ponieważ jest on językiem interpretowanym.

Patrząc na oba wyniki można zauważyć, że własna implementacja okazała się szybsza od biblioteki NetworkX niezależnie od badanego problemu. Pokazuje to, że w przypadku algorytmów grafowych wpływ na czas wykonywania ma nie tylko użyta reprezentacja, ale też narzędzie.

## 8. Przemyślenia i wnioski końcowe

Projekt ten, podobnie zresztą jak poprzedni, był bardzo ciekawym doświadczeniem. Same algorytmy nie były jakoś szczególnie skomplikowane pod względem implementacji, więc napisanie programu nie było problemem. Dodatkowo doświadczenie z używania biblioteki do parametrów, zdobyte przy poprzednim projekcie, pozwoliło na znacznie szybsze wymyślenie samej koncepcji uruchamiania programu i testów. Oprócz tego, wnioski wyciągnięte z drobnych problemów z poprzedniego projektu, pozwoliły na zaimplementowanie lepszego systemu logowania, który pozwalał wykryć błędy znacznie szybciej, przez co praca nad projektem nie była aż tak wyczerpująca.

Co do kursu, był on również bardzo dobry. Projekty nie były bardzo trudne ale przy wymaganiach na ocenę 5 (szczególnie przy pierwszym projekcie) trzeba było się trochę postarać. Nie oznacza to jednak że jest to złe – im więcej pracy tym bardziej rozwijają się umiejętności programowania co jest raczej na plus. Jeśli chodzi o najbardziej uciążliwą część to na pewno było to samo uruchamianie badań, które musiały być wykonywane godzinami, a następnie formatowanie ich i tworzenie odpowiednich wykresów. Co prawda można było tego uniknąć wybierając mniejsze rozmiary danych lecz wtedy różnice mogłyby nie być aż tak zauważalne więc coś za coś.

Jeśli chodzi o wyniki, potwierdziły one to, czego można było się spodziewać na podstawie teorii. Zarówno liczba wierzchołków grafu, jak i jego gęstość miały bardzo duży wpływ na czas działania badanych algorytmów. Wraz ze wzrostem rozmiaru grafu rosła również liczba wykonywanych operacji, co oznaczało wydłużenie czasu obliczeń. Podobnie wyglądała sytuacja w przypadku gęstości grafu – większa liczba krawędzi oznaczała konieczność przetworzenia większej ilości danych, co było widoczne w wynikach benchmarków.

Ciekawe było również zaobserwowanie, jak różne reprezentacje grafów potrafią wpłynąć na czas wykonywania algorytmów grafowych oraz sam fakt, że macierz incydencji w szybkim tempie potrafiła urosnąć do rozmiarów ponad 1400 GB, co sprawiało, że proces był zabijany przez system. Z tego powodu zaimplementowano ograniczenie dla tej reprezentacji i należy potwierdzić jej utworzenie, jeśli przekracza ona 4 GB.

Interesującą częścią projektu były także testy przeprowadzane na rzeczywistych datasetach. Pozwoliły one zobaczyć, jak badane algorytmy zachowują się nie tylko na sztucznie generowanych grafach, ale również na danych pochodzących z rzeczywistych scenariuszów. Bardzo miłym zaskoczeniem było tutaj też „pokonanie” gotowego rozwiązania jakim była biblioteka NetworkX, choć dla jeszcze większych grafów mogłoby dojść do sytuacji gdzie własna implementacja byłaby mimo wszystko niewystarczająca.

Podsumowując, projekt pozwolił na lepsze zrozumienie działania algorytmów grafowych oraz ich zastosowania w realnym świecie. Uzyskane wyniki były zgodne z oczekiwaniami, a

sam projekt był interesującą okazją do przypomnienia sobie wiedzy o grafach i zastosowania jej w praktyce.

## 9. Bibliografia

- [1] Malinowski, M. (2024, March 17). *Algorytmy Grafowe*. Informatyka na dziś - Algorytmy Grafowe. <https://www.drmalinowski.edu.pl/posts/2544-algorytmy-grafowe>
- [2] Wikimedia Foundation. (2025, December 20). *Graf (matematyka)*. Wikipedia. [https://pl.wikipedia.org/wiki/Graf\\_\(matematyka\)](https://pl.wikipedia.org/wiki/Graf_(matematyka))
- [3] Horzyk, A. (2022, January 21). *Wstęp do Informatyki - Grafy i struktury grafowe*. Akademia Górniczo-Hutnicza Wydział Elektrotechniki, Automatyki, Informatyki i Inżynierii Biomedycznej. <https://home.agh.edu.pl/~horzyk/lectures/wdi/WDI-Grafy.pdf>
- [4] Wikimedia Foundation. (2026, April 26). *Minimum Spanning Tree*. Wikipedia. [https://en.wikipedia.org/wiki/Minimum\\_spanning\\_tree](https://en.wikipedia.org/wiki/Minimum_spanning_tree)
- [5] Wikimedia Foundation. (2022, January 3). *Problem Maksymalnego Przepływu*. Wikipedia. [https://pl.wikipedia.org/wiki/Problem\\_maksymalnego\\_przep%C5%82ywu](https://pl.wikipedia.org/wiki/Problem_maksymalnego_przep%C5%82ywu)
- [6] Wikimedia Foundation. (2025, October 1). *Algorytm Prima*. Wikipedia. [https://pl.wikipedia.org/wiki/Algorytm\\_Prima](https://pl.wikipedia.org/wiki/Algorytm_Prima)
- [7] Wikimedia Foundation. (2026, January 17). *Algorytm Kruskala*. Wikipedia. [https://pl.wikipedia.org/wiki/Algorytm\\_Kruskala](https://pl.wikipedia.org/wiki/Algorytm_Kruskala)
- [8] Wikimedia Foundation. (2025b, December 20). *Algorytm Dijkstry*. Wikipedia. [https://pl.wikipedia.org/wiki/Algorytm\\_Dijkstry](https://pl.wikipedia.org/wiki/Algorytm_Dijkstry)
- [9] Wikimedia Foundation. (2026b, April 26). *Bellman–Ford algorithm*. Wikipedia. [https://en.wikipedia.org/wiki/Bellman%E2%80%93Ford\\_algorithm](https://en.wikipedia.org/wiki/Bellman%E2%80%93Ford_algorithm)
- [10] Wikimedia Foundation. (2026c, May 22). *Ford–Fulkerson algorithm*. Wikipedia. [https://en.wikipedia.org/wiki/Ford%E2%80%93Fulkerson\\_algorithm](https://en.wikipedia.org/wiki/Ford%E2%80%93Fulkerson_algorithm)
- [11] 9th DIMACS implementation challenge: Shortest paths. (n.d.). <https://www.diag.uniroma1.it/challenge9/download.shtml>